TOWARDS GLOBAL INTELLIGENCE FROM LOCAL DYNAMICS

Submitted to the Department of Computer Science of Amherst College

in partial fulfillment of the requirements

for the degree of Bachelor of Arts with honors

May 7, 2026

Author: Ivan Betancourt

Advisor: Prof. Lee Spector

Abstract

Neural Cellular Automata (NCA) are well-established vehicles of emergence and complexity, making them compelling candidates for general-purpose problem-solving. By design, NCA excel at tasks requiring local interactions but their capacity for global coordination and the methods by which we can improve it remain an active research area. This thesis explores the application of NCA to the structurally aligned Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) benchmark, and more specifically the effects of integrating various evolutionary approaches towards building global-level problem-solving abilities. This work investigates Gradient Lexicase Selection (GLS), a selection mechanism for evolutionary algorithms, as the primary training algorithm for NCA as well as an NCA refinement tool. Results indicate that fine-tuning NCA with GLS yields the greatest performance gains over baseline models. However, training NCA from scratch with GLS, though degrading performance on average, resulted in large performance gains on select tasks that the baseline NCA struggled on. We further demonstrate a statistically significant correlation between task example density and GLS performance improvement, suggesting that GLS is most effective in data-rich settings or when foundational representational priors already exist.

Acknowledgements

To Professor Spector, thank you for your unwavering support and enthusiasm towards my research throughout this process. It has been a great pleasure getting to work with and learn from you. To my current and former professors, thank you for making my undergraduate education so fulfilling and for fostering my love for learning. To my family and friends, thank you for being the strength and support I need to pursue the things I love with confidence. I am very grateful for you all.

Contents

Chapter 1

Introduction

The cells of a fetus cannot think. They cannot see, hear, feel, nor can they know anything about the greater system of which they are a part. They know only about what they perceive from their tiny neighborhoods, and yet they are able to meticulously arrange themselves into complex forms, unbeknownst to any single one of them. They form skin, layers of flexible, self-renewing, and waterproof fabric. They form the heart, a four-chambered pump with valves that open and close in synchrony, timed to milliseconds. They become the brain, a collection of folding tissue that will one day begin to reason about objects and ideas greater than itself and contemplate its own existence in the universe. From a mere interaction of cells emerges an impressively complex system. Morphogenesis, this process by which cells organize themselves into specific forms, is one of the most striking examples of the phenomenon computer science and machine learning researchers call emergence, and we observe it in countless other areas of nature. It is the process by which the complexity of a system reveals itself through the smaller and simpler interactions and processes that define it. Ant colonies transport food in neat single file lines and build intricate homes, while no one individual ever sees the greater picture. Starling murmurations dance in mesmerizing choreographies yet if you could ask a single starling where it is headed, it would tell you they are simply following the others. Even something as unnatural as the stock market exhibits emergent properties.

Cellular Automata (CA) demonstrate emergence in many different ways and serve as an ideal

testbed for studying it. A CA is a grid of cells that carry numerical values that are updated in discrete time-steps according to a deterministic rule. The quintessential example of the emergent complexity of CA comes from John Conway's Game of Life [6]. In Conway's infinite two-dimensional world, each cell is either dead or alive, and their state is determined by the following rule. A cell remains alive if two or three of its eight neighbors are alive. A cell dies if four or more of its neighbors are alive or if one or none of them are. A cell is born if it has exactly three alive neighbors. What seems like an unimpressive rule results in stunning complexity. When we let time pass unimpeded, Conway's world becomes populated with a wide range of life forms, each with their own unique behavior. The Blinker takes the form of a straight line of three alive cells with a perpetual spinning behavior. The Glider, when untouched by any other live cell, traverses the infinite expanse of the grid. The Pulsar, an alien-looking thing, appears to wave its eight tentacles menacingly. The emergent behavior of this simple CA is complex enough to declare it a Turing complete system. In fact, Paul Rendell demonstrated that it is possible to create a Turing machine inside the Game of Life [16]. Even more impressively, enthusiasts have managed to create the Game of Life *inside* the Game of Life. Another example of the emergent behavior of CA comes from Stephen Wolfram's work with the one-dimensional variant [22]. The cells of Wolfram's grid lie on a single line. He examines the behavior of different update rules by displaying their several states in successive rows, with each new row representing the next step of the automaton's evolution. Some of the rules Wolfram explored exhibit simple, predictable, and boring behavior, while others reveal shocking complexity. Rule 168 is a boring example; after only a few time-steps, all of the live cells in the initial generation die off. Rule 90 is as simple and deterministic as the rest, but a single live cell in the initial state generates the Sierpinski triangle. Rule 30 produces behavior indistinguishable from randomness without any stochasticity programmed into the rule. Martinez et al. showed Rule 110 to be Turing complete [12].

These examples demonstrate that CA are not just mathematical curiosities, but systems capable of producing structured and complex behavior. If Turing complete systems can emerge from simple local interactions, it stands to ask if systems with such properties can be engineered to solve problems. Indeed, CA have long been studied in this context. CA have been found to be capable of

generating secure session keys, segmenting images, and even predicting the movement of cancer cells [24, 1, 13]. The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) presents a particularly compelling challenge for CA. It is a benchmark designed to measure the ability of artificially intelligent systems to perform tasks that require the abstraction capabilities of humans. More specifically, it is a collection of transformation tasks defined on finite grids of cells that provide only a few examples for each task, requiring the solver to learn how to solve problems from sparse data. Using CA to solve the ARC-AGI problems would require the manual development of an update rule that works for all of them. However, given the distinct needs of each task, the search space of such programs is massive and navigating it by hand is infeasible. If CA are to be practical solutions to ARC-AGI, we need a method of defining their update rules autonomously.

Neural Cellular Automata (NCA) operate in the same way as their non-neural counterparts but with an added component. In an NCA, the update rule is learned with machine learning methods rather than being explicitly programmed. More concretely, the update rule becomes a differentiable function parameterized by a neural network that can be trained to minimize some loss function. These parameters can be tuned with optimizer algorithms just like any other neural network. This extension to CA allows for a greater level of complexity and computational capability to emerge. Mordvintsev used NCA to successfully model morphogenesis [15]. Using a variety of low-resolution images, his models were trained on perturbations of those images in order to learn how to regenerate the full image, and they do so impressively well. This is a promising result for the case of problem-solving CA. While gradient-based learning may produce decent results, it is not without faults. Gradient Descent, a standard neural network optimizer, works best with a smooth loss landscape and can thus struggle to converge on strong solutions in problem domains with multiple objectives and sparse supervision. These limitations motivate the search for alternative optimization paradigms that rely on more than just gradient information.

One of the greatest optimizers observed in nature is Darwinian evolution. Through the repeated process of variation and selection, evolution produces remarkably fit organisms without an explicit plan to do so. Importantly, evolution does not optimize for a single objective. Instead, organisms

are subject to many competing pressures simultaneously, such as survival, reproduction, robustness, and adaptability, none of which can be reduced to a single measure of fitness. This observation has motivated decades of work in the field of Evolutionary Computation, where similar principles are applied to the optimization of computational systems. In these approaches, candidate solutions are iteratively modified and selected based on their performance on one or more tasks, allowing complex behaviors to emerge even when the search space is poorly understood. Evolutionary algorithms have proven to be especially effective in settings where success requires satisfying many constraints at once rather than excelling at a single objective. ARC-AGI is a great example of such a setting. Each ARC-AGI task can be viewed as a distinct objective, with success on the whole benchmark therefore requiring a system to perform well across a diverse collection of objectives. Optimizing a single error metric aggregated across all tasks risks favoring solutions that perform adequately on some tasks while failing catastrophically on others. This makes ARC-AGI a natural fit for selection mechanisms that explicitly account for multiple objectives. Lexicase Selection (LS) offers a compelling approach in this context. Rather than selecting individuals based on an averaged performance metric, LS evaluates candidates by considering tasks one at a time in random order, filtering the population based on the performance of each individual on each task independently [19]. This allows individuals who excel at rare or difficult tasks to be preserved, even if they perform poorly on others.

In this thesis, we explore the application of Gradient Lexicase Selection (GLS), a variant of LS, to the evolution of NCA trained to solve the ARC-AGI tasks and aim to answer whether evolutionary pressure applied at the level of gradients can guide NCA toward more abstract and robust problem-solving behavior. Like prior work in this area, we will primarily investigate training a separate model for each task. In addition to this, we explore the use of GLS as a refinement tool and the loftier goal of solving the ARC-AGI benchmark with a single NCA trained on all of its tasks.

Chapter 2

Background

2.1 Evolutionary Computation and Genetic Algorithms

Evolutionary computation is founded on the principles of Darwinian evolution, the process that gave giraffes their long necks and made feathers so effective for generating lift. It is this process that makes it seem as though the features of the world were carefully designed for the organisms that inhabit it. Of course, this is not the case; evolution, wielding the paintbrush of natural selection, is the designer of organisms. Natural selection is the driving force behind why organisms possess the characteristics they do. It is not a coincidence that sweat cools us off or that we can hardly distinguish between a leaf and a Phylliidae insect; it is a selection. The main principle of natural selection says that the traits which increase the chance of survival are those which will spread throughout the population for the simple reason that the organisms with those traits are still alive to pass them down. Through random genetic mutations, new traits are introduced into the population and have the opportunity to be selected for or not. This undeniably effective approach to creating strong solutions (organisms) to problems (the hostile Earth) has garnered the attention of computer scientists since the 1950s and inspired them to harness its power. They wondered, what if instead of manually crafting solutions to problems, we let the solutions evolve themselves over many generations?

John Holland proposed a way to do just that in the early 1970s. His approach eventually became

known as the Genetic Algorithm (GA). Such algorithms are composed of four primary elements:

1. **Population:** A set of candidate solutions.
2. **Fitness Function:** A metric for evaluating solutions.
3. **Selection Procedure:** An algorithm that chooses the best solutions.
4. **Reproduction and Mutation Procedure:** An algorithm that builds a new population.

GAs seek to model the evolutionary process. At a very high level, one or more of the fittest solutions are selected at every generation to be combined and mutated in order to produce a new population of solutions. If there were a perfect isomorphism from GAs to the real world, it would map solutions to organisms and the combination of those solutions to reproduction. But no such isomorphism exists, and the primary reason is that fitness in nature cannot be restricted to a single scalar value, as the idea of a fitness function would like to suggest. We use fitness functions in GAs to determine which solutions we would like to consider in the subsequent generation. This approach makes it easier for us to compare solutions and decide which is better, allowing us to sidestep the reality that fitness in the real world is determined by a massive combination of factors, many of which are unquantifiable. Later, we will explore a technique that gets us closer to a perfect isomorphism.

Solutions in the canonical GA are represented as bit strings, e.g., 11001, similar to how DNA is a sequence of chemical bases [21]. Not only can these bit strings encode any set of parameters, but they are also easy to manipulate, a property that will prove useful for reproduction and mutation down the line. Each bit, or group of bits, maps to a specific parameter of the solution, retaining all the information necessary to represent it. To further ground this idea, suppose we have a very simple problem which we will call **31**. **31** asks "given any integer x between 0 and 31 inclusive, is x equal to 31?" Although a GA may be overkill for such a problem, it will nicely illustrate all of its moving parts. To begin, we might consider the following initial population of positive integers:

$$P = \{1, 25, 3, 11, 10, 7\}$$

These numbers have been generated randomly. A natural choice for representing these numbers as bit strings is simply to use their binary equivalents. In order to ensure that all solutions have the same format and are easy to compare, all strings will have a length of 5:

$$P = \{00001, 11001, 00011, 01011, 01010, 00111\}$$

Since 11111 is the binary equivalent of 31, the problem now becomes "is the bit string 11111 present in our population?" Currently, the answer is no, but it does seem like we are close. Both 11001 and 00111 are just two bits away from being the correct solution. We can quantify this notion of closeness to the correct solution with a fitness function. Consider the following function.

$$f(x) = \sum_{i=1}^{5} x_i \text{ for any bitstring } x \text{ of length } 5$$

This function simply returns the sum of the digits in the given bit string. For example, $f(01000) = 1$ and $f(11011) = 4$. We can easily apply this function to problem **31**; a bit string x is the correct solution to **31** if and only if $f(x) = 5$. Hence, the closer $f(x)$ is to 5, the better x is as a solution. We now have a quick and interpretable method of deciding which solutions are superior to others, and, in particular, which solutions we would like to keep in our population.

Arguably, the most important step in a GA is the selection procedure. Just as natural selection is the designer of organisms, the selection procedure of our GA is what will determine the characteristics of our solutions. Researchers have experimented with various approaches to designing this procedure, each with their own unique characteristics and philosophies. To introduce this topic, we will apply a very widely used selection mechanism to **31** called Tournament Selection. The goal of the selection phase in GAs is to choose which solutions will become the parents of the next generation. Hence, the next generation of solutions is generated from a subset of the current generation, and it is the selection procedure's job to build that subset. Tournament Selection builds that subset as follows: Randomly select t solutions from the population with replacement and place the solution with the best fitness value into an intermediate population which we will call the mating pool, and repeat N times [3]. Let the mating pool be denoted by M and run Tournament Selection

on our population P three times with $t = 2$:

1. Random Selections: $\{00011, 00001\}$. Result: $M = \{00011\}$ as $f(00011) > f(00001)$.

2. Random Selections: $\{11001, 01010\}$. Result: $M = \{00011, 11001\}$ as $f(11001) > f(01010)$.

3. Random Selections: $\{00111, 00011\}$. Result: $M = \{00011, 11001, 00111\}$ as $f(00111) > f(00011)$.

The resulting mating pool is $M = \{00011, 11001, 00111\}$ and these solutions will become the parents of the next generation. This instance of Tournament Selection seems to have performed well; the average fitness value of M is about 2.67, 0.47 fitness units higher than that of P, our original population. However, it is conceivable that the procedure may not have performed so well if luck was not on its side. Since Tournament Selection makes random selections at every iteration, it is possible that, by pure chance, it chooses low-fitness solutions, resulting in an average fitness value of the mating pool that is actually worse than the average of the original population. We might consider increasing the tournament size, indicated by the variable t, to increase our chances of choosing fitter solutions at each iteration. Indeed, this change is at the heart of what makes Tournament Selection a strong selection procedure. When we increase the tournament size, what we effectively increase is the selection pressure [14]. Selection pressure is the ability of a selection mechanism to favor the fittest solutions, and it is a critical component of GAs. It is what drives a GA towards the correct solution; without selection pressure, the algorithm is an aimless population-generating machine. However, creating an effective GA is not all about cranking selection pressure to the maximum. There is an inherent trade off in increasing selection pressure and this will become more apparent as we explore Lexicase Selection. But to offer some basic intuition for this, filtering for only the best of the best solutions shrinks the search space of solutions considerably, potentially blocking us from great solutions that may live very far away in that space.

To tie the bow on our GA, we must now generate a new population from the selected parents, which in our case live in the mating pool. There are many ways to do this, but a simple method is 1-point crossover. At each step, we randomly choose a pair of solutions from M, and with a

predefined probability p, we combine them and add the resulting offspring to our new population. 1-point crossover randomly chooses a combination point, i.e., an index, on which to concatenate the segments of the bitstrings resulting from splitting them at that point [21]. For example, suppose we randomly select to combine 00011 and 11001 at the combination point 3. We split 00011 at index 3 to get segments 000 and 11. Similarly, we split 11001 to get segments 110 and 01. We then combine the head of 00011 with the tail of 11001 and vice versa, resulting in new bitstrings, 00001 and 11011. Each combination thus results in two offspring. In the case where the randomly sampled value exceeds the probability threshold p, the selected pair of solutions are added to the new population as they are. Mutation, as in nature, introduces unseen traits into the population. This is often implemented as a low-probability bit flip, though many other approaches exist. Over many generations, the cumulative effect of selection pressure and genetic variation drives the population toward convergence on high-fitness solutions.

2.2 Lexicase Selection

Lexicase Selection (LS) presents a compelling way to manage the balance of selection pressure in GAs. To motivate LS, we must first look beyond trivial problems like **31**. Many of the problems that engineers and researchers work on in the real world are, of course, nothing like **31**. Yes, in part because the problem is painfully simple, but also for a deeper reason: There is only one goal to **31**. We do not care whether a solution to **31** is prime or odd or positive; we only care whether it is equal to the quantity 31. The kinds of problems that we will focus on in this thesis are opposite to **31** in this respect. Often described as modal or multi-objective, these problems require a solution to be effective in multiple scenarios. Consider the challenge of calculating a formula for determining some measure of a given geometric shape [18]. We could imagine the following inputs for this problem: (Perimeter, Pentagon), (Surface area, Sphere), (Volume, Cone), and so on. The correct solutions to these inputs are qualitatively different from each other. Therefore, the method for finding these formulae needs to be effective in doing so for a wide range of distinct cases. Consider evolving a program as an approach to solving this problem; this is where we must begin to reconsider how we

interpret the fitness of a solution, as we can no longer correlate an increase in a single fitness score with progress towards every objective. LS, a selection procedure developed by Lee Spector, thrives in this setting.

Deviating from the trend of fitness aggregation in GAs, LS uses each objective in the given problem as a test case for solutions and evaluates them on each case one at a time. More specifically, using a randomly ordered sequence of these test cases, LS eliminates the solutions from the population that do not have the best fitness on the first case. If more than one solution satisfies this, we do the same on the second test case. This process continues until only one solution remains or until all test cases have been exhausted. If more than one solution remains in the population after the test cases are exhausted, a solution is selected at random. The complete procedure is detailed in Algorithm 1.

Algorithm 1 Lexicase Selection

```
procedure LexicaseSelection(candidates, cases)
    shuffled_cases ← Randomly shuffled cases
    for each case in shuffled_cases do
        Evaluate all candidates on case
        candidates ← The subset of candidates that have exactly the best performance
                     on case
        if candidates contains only a single candidate then
            return candidate
        end if
    end for
    return Randomly selected candidate from candidates
end procedure
```

The strength of LS comes from the kinds of solutions it chooses. Let us revisit our GA from the previous section and apply it to evolving a program for the geometric-formulae problem. We could imagine that in this case, a fitness function might measure how close the returned value of an evolved program is to the value that its input expects. There are, however, many possible inputs

to this problem, each requiring different behavior, so we should account for this in our fitness calculation. The conventional approach is to take some aggregate of the fitnesses across all the possible input cases, so let us choose to take the mean of the fitnesses. Recall that our GA used Tournament Selection to select a parent for the next generation. So when it comes time to select it, we will sample random subsets of the population and choose the solutions with the highest fitness; in this case, the solutions with the best average performance over all cases are the selected solutions. Therefore, we only ever choose solutions that perform adequately on all the cases and ignore outliers. This may seem like a promising strategy, but let us now consider what happens when we use LS instead. Since we consider each case individually, we no longer need aggregation. Suppose the first few elements of the test case sequence are (Perimeter, Pentagon), (Surface area, Sphere), and (Volume, Cone), in that order. First, the solutions that are not the absolute best at computing the perimeter of a pentagon are eliminated from consideration. Suppose two solutions remain after this. We now choose the solution that is the best at calculating the surface area of a sphere. Assume now that only one solution satisfies this. The procedure has now ended and we have chosen a somewhat peculiar parent. This solution, relative to all the others in the population, is great at finding both the perimeter of a pentagon and the surface area of a sphere, but for all we know, it could perform catastrophically in other cases. As we repeat the LS process to select more parents, we build a set of parents that is composed of "specialists": Solutions which perform particularly well on a subset of the cases but worse, if not terrible, on the rest. If we were to average the fitness of these solutions across all cases, the resulting value would almost always be less than that of the solutions selected by Tournament Selection. Why, then, would we want to choose these specialist solutions over the well-rounded ones? The primary reason for this is the nature of the problems LS is known to solve well, namely, uncompromising problems [10]. An uncompromising problem, as coined by Helmuth and Spector, is a multi-objective problem where we do not tolerate compromises across cases and instead require good performance across the board. Although this may seem like a small subset of problems, many of them share this characteristic. The geometric-formulae problem is an example of such a problem. Can a solution that is great at all things circular but useless for polygons be considered a good solution? Likely not. A good solution to this problem

would not make any trade-offs anywhere. This is what makes the problem uncompromising. The problem domain that we will study in this thesis is also uncompromising, serving as the primary motivation for the use of LS.

In addition to being an ideal fit for the problems we will work with in this study, LS possesses a beautiful resonance with nature. Selection in nature never plays out as it does in GAs, although LS gets us closer to that ideal. Being an organism on Earth means being subject to the countless highs and lows that come with living life. It means you will get hurt, learn new skills, grieve your loved ones, see the beauty of the world, age, and fall in love. Whether or not you reproduce in this life is not determined by a single number, but rather by how you handle the series of challenges and gifts life presents to you.

2.3 Machine Learning and Neural Networks

Machine learning is typically applied to problems that might be simple for humans to solve but which are sufficiently complex so as to be too difficult to *explain* how to solve. For instance, consider the image classification problem. It takes barely any mental effort for us to classify what any given image is of. If we see a cat, then the image is of a cat. If we see a tree, then the image is of a tree. If we see a bunch of binary strings being filtered and combined ad nauseam in a diagram, then we might infer that the image is a depiction of a GA. Machines, however, have no concept of cats or GAs. If we wanted to make a computer identify what is contained in an image, the millions of if-statements we might write in a program to solve this problem would not get us any closer. For one, the number of statements in such a program would be beyond intractable. For each if-statement, there could be millions of other conditions that the machine might need to consider in order to make the right prediction. Even more problematic, it is not at all obvious what these statements should even convey. When you, as a human, classify an image of a tree, what aspects of the image do you rely on to make your deduction? The color of the leaves? The number of branches? Perhaps the context surrounding the tree? Well, it is hard to say; all of these features may vary widely across different images. In reality, there are thousands of qualities of the image

that our brains have learned over the years to associate with trees, some of which we may not even be aware of. All of this to say, there are some problems which are infeasible to teach a machine how to solve, and in these cases, the machine is better off learning how to solve them on its own. This is the essence of machine learning. Instead of manually crafting a program to instruct a machine on how to solve a problem, we can use data and function optimization to make the machine learn how to do it itself.

The quintessential example of how machine learning works in practice is the neural network. A neural network is generally made up of many connected multilayer perceptrons. Therefore, the multilayer perceptron serves as a representative model to illustrate the fundamental principles of how neural networks learn from data. At its core, a multilayer perceptron is a mathematical formula that produces a numerical output given numerical inputs. Given inputs x_1 and x_2, and variables w_1, w_2, $\ldots$, w_6, and b_1, b_2, b_3, which we call weights and biases respectively, the multilayer perceptron depicted in Figure 2.1 computes the output o with the formula

$$o = f(w_5 h_1 + w_6 h_2 + b_3) = f(f(w_1 x_1 + w_2 x_2 + b_1) + f(w_3 x_1 + w_4 x_2 + b_2) + b_3)$$

where f is what is called an activation function. The role of the activation function is to introduce non-linearity into the model by deterministically altering the value in each node. Otherwise, the network would only be able to learn linear relationships between inputs, as each node is indeed just a linear combination of weights and biases. So, if we have a defined set of weights and biases, we can use this multilayer perceptron to map the input data to a final prediction. But how do we know how to define these weights and biases? Backpropagation and Gradient Descent are the drivers behind how machine learning models learn these values.

To start the training process, we assign random values to all the weights and biases. After using these weights and biases to make a prediction, we can compute a loss function which quantifies the error between the predicted value and the target value. Backpropagation then allows us to calculate exactly how much each individual weight and bias contributed to the computed error with gradients from differential calculus. Once all gradients are calculated, Gradient Descent steps in to update each parameter according to those gradients. It nudges each weight and bias in the opposite

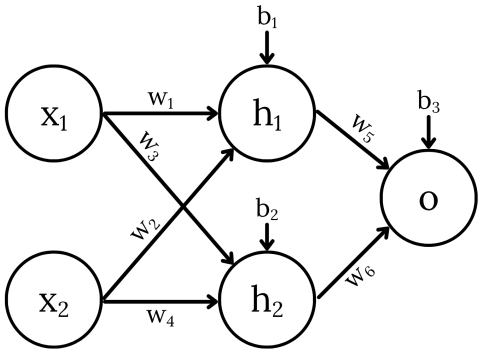

Figure 2.1: Multilayer Perceptron

direction of its gradient to minimize the error. Exactly how much each weight and bias is nudged is determined by the specific optimizer algorithm in use. A popular optimizer algorithm which is also relevant to the experiments of this thesis is Stochastic Gradient Descent (SGD). Standard Gradient Descent looks at the entire dataset before making adjustments to any parameters. This is accurate but incredibly slow and memory-intensive. SGD, on the other hand, looks at a small, randomly sampled subset of the data from which to calculate the gradients and uses only those to determine how much to modify the weights and biases. Through thousands of these tiny adjustments across many iterations, the entire network converges on parameters which optimize for minimal error and, in turn, high prediction accuracy.

2.4 Neural Cellular Automata

Neural Cellular Automata (NCA) represent an advancement to the standard CA with machine learning. The local and emergent properties of CA make them compelling candidate solutions for select problems across various fields, but particularly for the central problem of this thesis. However, if the given problem does not define an update rule for us or if there is insufficient information to deduce one ourselves, NCA become the natural approach since they can learn the rule themselves. In order to understand how NCA work under the hood, we must first understand Convolutional Neural Networks (CNNs). CNNs are great at solving the image classification problem partly because of how they perceive images. Standard neural networks flatten images into long arrays of numbers, each

number corresponding to a pixel. CNNs, on the other hand, preserve the image's two-dimensional structure. That said, the image is only really two-dimensional to us. CNNs actually perceive images as three-dimensional tensors: One dimension for height, one dimension for width, and the last dimension for channels (we use the word tensor in this thesis to mean a multi-dimensional array). The channels of an image typically represent color, but generally serve as a dimension of information. For example, images often have three channels to describe an RGB value for each pixel, but there is sometimes an extra channel to describe the opacity of the pixel.

The most remarkable piece of CNNs is how they perform feature extraction. At their core, CNNs are a collection of small 3D tensors of numbers, known as filters, which perform convolution, a mathematical operation, on portions of the image tensor one at a time by sliding those filters across it. As they move, they analyze and identify local patterns, enabling the network to detect important features. The numerical values contained by these filters, better described as weights, are what CNNs learn to optimize over time. To better illustrate this idea, suppose we have an $n \times n$ image $\mathcal{X}$ which uses the RGB coloring model. When we pass this image through a CNN, it perceives it as an $n \times n \times 3$ tensor of numbers. Suppose our filters of the CNN are $k \times k \times 3$ tensors of numbers, for some $k < n$. At this stage, our goal is to produce a new tensor which encodes a higher-level representation of the image's features by convolving each filter with every $k \times k \times 3$ patch of the image. We call the resulting object of convolution between a *single* filter and all $k \times k \times 3$ patches of the image a feature map. As will soon be made clear, the dimensionality of any feature map is always 2. Hence, performing convolution with *multiple* filters gives us our desired new tensor when we stack all the resulting feature maps on top of each other. But what exactly is convolution? Convolution between such a patch of $\mathcal{X}$ and a filter ϕ can be expressed as

$$y_{i,\,j} = \sum_{c=1}^{3} \sum_{m=-1}^{1} \sum_{n=-1}^{1} \mathcal{X}_{i+m,\,j+n,\,c}\ \phi_{m,\,n,\,c}$$

where $y_{i,j}$ is the pixel at row i and column j of the feature map corresponding to filter ϕ. This operation can be described as the sum of the element-wise products between the $k \times k \times 3$ patch

of the image centered at $\mathcal{X}_{i,j}$ and the filter ϕ. If you like, it is essentially the dot product between both objects. As such, this operation returns a single scalar and the 2D feature map is thus constructed by centering ϕ on the remaining pixels in the image and running convolution for each one. Centering the convolution around the pixel $\mathcal{X}_{i,j}$ is more a mathematical convention than it is a functional requirement. Describing convolution in this way lets us cleanly map pixels of the image to pixels of the feature map. However, this convention does highlight a problem at the edges of the image. When we want to perform convolution with a filter and a patch of the image centered at $\mathcal{X}_{0,0}$, for example, some of the pixels in the 3×3 neighborhood of $\mathcal{X}_{0,0}$ are not pixels at all; they are empty space outside the borders of the image. The same is true for every pixel on the edge of $\mathcal{X}$ and potentially for more depending on the size of k. For this reason, before we perform convolution, we often add a border around our image, known as padding, composed of pixels with value 0. Once convolution with all filters has concluded and the stack of feature maps has been built, a second layer of convolution can ensue. After that, a third can start, and so on. This process can be repeated indefinitely, each layer extracting finer grained representations of the image for the model. Figure 2.2 visualizes the construction of a feature map. After all layers of convolution have been exhausted, we flatten our stack of feature maps into a one-dimensional array and pass it into a standard neural network which makes a prediction from the given data. Depending on how far away that prediction is from the target prediction, Backpropagation and Gradient Descent optimize the filter weights, and the process repeats for many epochs.

NCA and CNNs share a fundamental connection through the locality of their dynamics and the temporal updating of pixels. This enables us to implement NCA as recurrent CNNs. We will first demonstrate how a CNN composed of a convolutional layer followed by a series of 1×1 convolutional layers can model the dynamics of any CA [7]. To be clear, when we refer to an $A \times B$ convolutional layer, we are expressing that the filters used in that layer are of the shape $A \times B \times C$ where C is the number of channels of the previous layer's output tensor. We will use the Game of Life for this demonstration. Recall that the Game of Life operates on a binary grid, so any input grid is accurately represented by a 2D $H \times W$ array. This means that our input grids have only one channel per pixel. Since the update rule for every pixel depends on the pixels within its 3×3

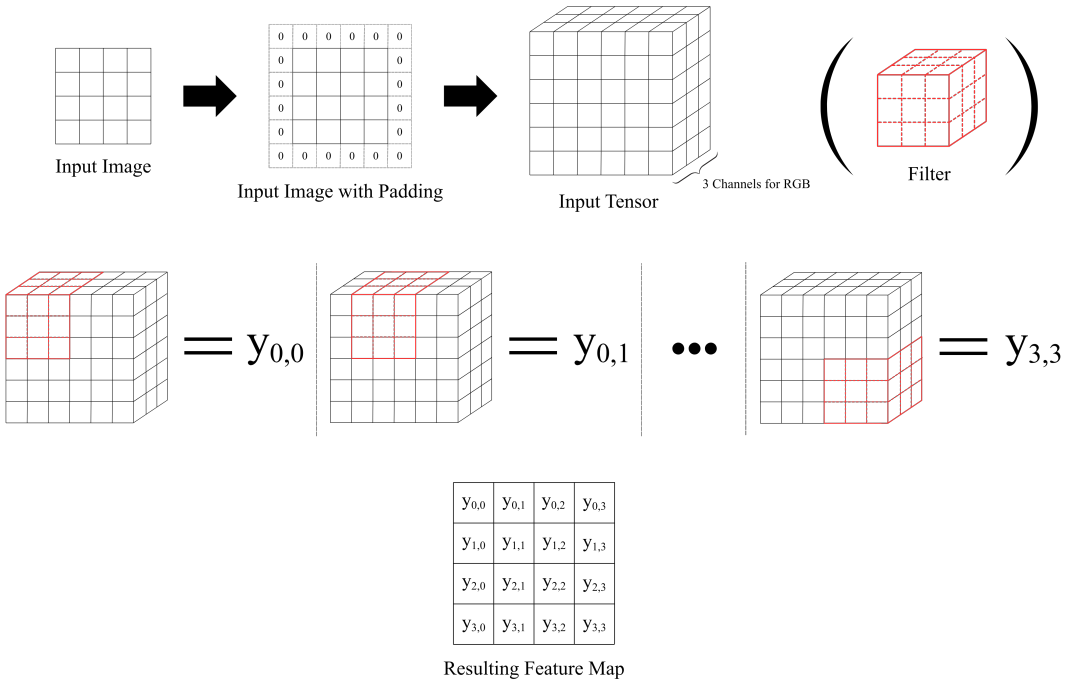

Figure 2.2: Visualization of Convolution Between an Image and Filter

neighborhood, our filters will be 3×3. Suppose these filters already encode the Game of Life update rule and that we have n of them. Just as any CA applies the update rule to every pixel, we perform convolution between our filters and every 3×3 patch of the input grid, assuming a padding width of 1. As we covered earlier, convolution between one filter and all patches of the input results in one 2D feature map, so convolution with all of our n filters results in an $H \times W \times n$ tensor. All that is left is to consolidate this tensor into a 2D grid. We can do this by passing the tensor through a 1×1 convolutional layer with a single filter. Once convolution is finished, we have effectively modeled one time-step in the Game of Life with a CNN. But what if our 3×3 filters did not yet encode the Game of Life update rule? This is where learning comes in. If we have some state of the Game of Life S_t, we can compare the output of the CNN with the target state S_{t+1}, compute the loss, and run Backpropagation and Gradient Descent. In this case, we are training the model on the particular transformation $S_t \rightarrow S_{t+1}$. We can take this a step further. If we repeatedly

pass the output of the CNN back into itself, say m times, then we could instead compare the final output with the target state S_{t+m}, effectively testing the transformation $S_t \to S_{t+m}$. When we take the loss here, we perform a special kind of Backpropagation called Backpropagation-Through-Time where gradients are calculated across multiple calls of the network. This is why we call an NCA a recurrent CNN. A recurrent neural network specializes in processing data that evolves over time by using its output from the previous state as the input for the current step. Typically, we include several more 1×1 convolutional layers as they have been shown to cheaply increase the representational power of the network [20]. One way to understand this is to think of the 1×1 layers as multilayer perceptrons. Recall that a multilayer perceptron essentially multiplies each element by a weight and adds them together repeatedly. This is exactly the operation performed by a 1×1 convolutional layer. Therefore, each 1×1 layer is like its own neural network that helps the NCA further learn and refine the rule.

2.5 ARC-AGI

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) is the problem domain around which this thesis is centered. ARC-AGI is a benchmark meant to test the limits of the intelligence of various systems, particularly aimed towards measuring the degree to which these systems can complete tasks with generality [4]. Though there are many perspectives on what the true definition of intelligence is, Francis Chollet, the developer of this benchmark, has put forward his own definition that aims to be actionable and enable us to more holistically evaluate the cognitive abilities of our systems: "*The intelligence of a system is a measure of its skill-acquisition efficiency over a scope of tasks, with respect to priors, experience, and generalization difficulty*" [4]. Expanding on this, he argues that given two systems which have similar knowledge priors and which are exposed to the same unseen tasks, the system with greater intelligence is the one that has learned greater skills. The design of ARC-AGI is meant to capture exactly this.

ARC-AGI is structured as a set of transformation tasks defined on 2D grids of integers from 0 to 9. When visualized, the grids are typically displayed as grids of colored pixels, one color per

integer, as shown in Figure 2.3. Each task can be described by two such grids: One for the input and the other for the target output. There are an average of approximately 3 examples across all tasks. That is, about 3 input/output pairs of grids that demonstrate the intended transformation. These examples are made available to solvers for them to learn from. In addition to these examples, there is an extra pair for each task that is meant to serve as the test; solvers are not to look at this example before evaluation time.

The tasks in ARC-AGI range from simple color maps, e.g., replacing all red pixels with blue, to more complex tasks like geometric extrapolations, such as extending a line segment along its angular trajectory until it intersects the grid boundary. Given a few minutes, if not seconds, a human will be able to infer the intended transformation to successfully solve the task. In contrast, developing computational solutions to the benchmark has remained a significant challenge. Many such attempts have been made, most of them leaning on machine learning methods. Given the recent rise in popularity of artificial intelligence and excitement surrounding its capabilities, many large language model providers such as OpenAI, Anthropic, and Google take pride in their performance on the ARC-AGI benchmark. Though they are indeed able to solve many of the tasks, the ARC-AGI team reports a clear correlation between the percentage of tasks solved and the dollar cost per task, nearing $100 per task. It should be noted that as of the completion of this thesis, ARC-AGI remains unsolved. Given the nature of ARC-AGI and the architecture of NCA, we are inclined to believe that NCA present a more natural and cheaper alternative to solving the benchmark.

2.6 Neural Cellular Automata for ARC-AGI

Guichard et al. established NCA as viable solvers of the ARC-AGI tasks, highlighting their competitive performance and cost efficiency [8]. They used standard NCA as well as variants of EngramNCA, an NCA architecture featuring hidden memory states geared towards storing "genetic" morphologies [9]. In a similar study, Xu and Miikkulainen examine the characteristics of NCA solutions for the ARC-AGI tasks [23]. They introduce a stochastic updating regime for their NCA

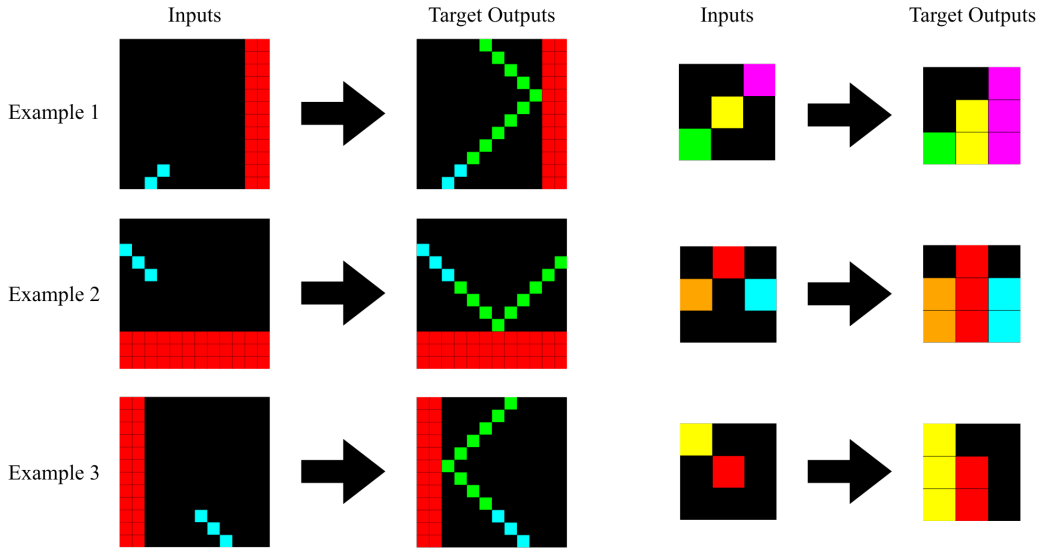

Figure 2.3: Training Examples of Tasks 508bd3b6 and d037b0a7

which they claim to act as a powerful regularizer, preventing weak models which overfit to the training examples and encouraging the development of robust transformation rules. They further discuss the shortcomings of their NCA by analyzing various failure cases. A study conducted by an independent researcher under the alias Sasmitha overlaps the most with our work [17]. They use an ensemble of NCA, evolved without Backpropagation, to solve the ARC-AGI tasks, resulting in 16.11% of the tasks correctly solved.

Chapter 3

Experimental Design

3.1 Replication Study

The NCA architecture that we study in this thesis mirrors that in [23]. As such, we begin by replicating their architecture and validating their results to establish a reliable baseline for future comparison. As we covered earlier, the ARC-AGI tasks are composed of 2D grids of integers from 0 to 9. Before training, we convert these grids to 3D tensors where each cell holds a value of 0 or 1 with each channel representing a unique color. More specifically, an $H \times W$ ARC-AGI task is converted into an $H \times W \times 10$ tensor where each cell has a value of 1 only in one of the 10 channels and zeros in the rest. This tensor is what our NCA will come to know as a task. As a hyperparameter to our NCA, we add a predefined number of hidden channels to the tensor. These extra channels are "hidden" because we do not impose that they encode any particular meaning; they are free to represent any kind of information that might be helpful for solving the task. The NCA expects 20 hidden channels, so it takes as input and produces as output tensors of shape $H \times W \times 30$. We infer the predicted color of a particular pixel to be the $argmax$ of the first 10 channels. For instance, if the value of the cell (i, j, 7) carries the largest value of those in its channel, i.e., the values of the cells (i, j, 0 ... 9), then we say the NCA has predicted a value of 7 for the cell (i, j) in the final 2D output grid.

Our NCA are made up of three convolutional layers: One 3×3 layer and two 1×1 layers. With

a padding of 1, the filters of our 3×3 layer, of which there are 30 (one for each channel), sweep through the tensor and perform convolution with every $3 \times 3 \times 30$ patch to produce a stack of 30 $H \times W$ feature maps. With this alone, we effectively model a CA with an update rule determined by the 3×3 neighborhood of its cells. The 1×1 layers help refine the learned rule even further. Once the 3×3 layer produces its output tensor, represented by the stack of its feature maps, we append the first 10 channels, i.e., the color channels, of the initial pre-convolution tensor to this output tensor along the channel dimension and pass it into the first 1×1 layer. In this way, we explicitly give the 1×1 layer access to the initial color distribution in addition to the neighborhood-derived features, which helps to learn relationships between a cell's individual state and its surrounding spatial context. This necessarily requires the filters in this layer to have shape $1 \times 1 \times 40$ given the concatenation of 10 extra channels. There are 30 of them, meaning that the tensor is restored to its original shape after all convolution is done. We pass the output tensor of this 1×1 layer into the final 1×1 layer and then return the color channels of the final layer's output tensor after applying a $softmax$ transformation on them, allowing us to interpret the result as a probability distribution of colors for each pixel and more readily compute loss metrics. During training, we feed the resulting tensor back into the network for a total of 10 iterations. For each of these iterations, the tensor is fed into the NCA 128 times as done in [23] to obtain a stronger gradient estimate. Figure 3.1 illustrates the full process.

An important piece of this NCA architecture is asynchronicity. Though many CA update their cells simultaneously, we believe synchronicity is more of an implementational convenience than a necessary feature of dynamical systems. In fact, dynamical systems in nature are almost never synchronous. Asynchronicity in [23] is implemented by randomly selecting a portion of cells to update at each step. At the end of each update step, each cell is replaced with a combination of itself and the corresponding cell in the previous state with probability 0.75. This combination is implemented as a convex combination, where a strength weight $m \in [0, 1]$ is sampled per cell and the resulting cell is determined by $(1 - m)\, S_t(i,\, j) + m\, S_{t-1}(i,\, j)$ where $S_t(i,\, j)$ is the value of the cell at $(i,\, j)$ in the newly computed state S_t. At evaluation time, we run the trained models with synchronous updates for consistency and stability.

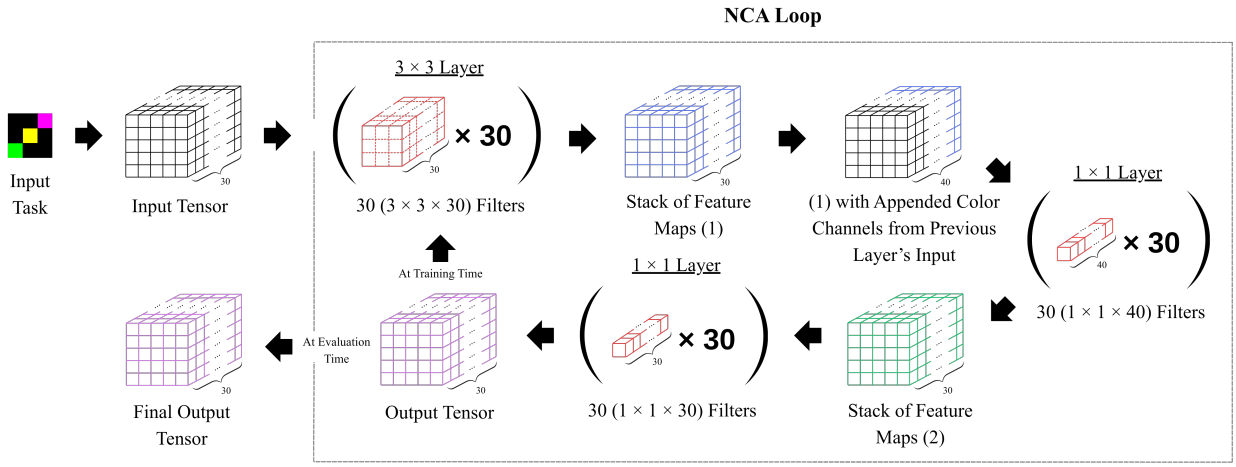

Figure 3.1: Forward Pass Visualization of a Task Through NCA

Xu and Miikkulainen trained and tested a separate NCA on each task in the ARC-AGI dataset. We replicated this experiment and found the same success rate of 13.37% of tasks fully solved. It is important to note that not every ARC-AGI task is suitable for our architecture and was thus not included in this experiment and the ones to come. Our NCA features no mechanism for changing grid sizes, so we restrict our experiments to tasks which maintain the same shape across inputs and outputs. Moreover, some tasks feature colors in the test example that do not appear in the training examples. Since colors represent distinct channels in our NCA, their performance on such tasks, which we would expect to be very poor, is not indicative of the strength of our approach. Excluding these kinds of tasks leaves us with 172 valid tasks.

3.2 Single-Task NCA

We are ready to investigate how evolution affects the performance of NCA on the ARC-AGI dataset. We will use the models from our replication experiment as our baseline to which we will compare our evolved models, hereafter referred to as Single-Task NCA. We implement the evolution of our NCA with a variant of LS called Gradient Lexicase Selection (GLS), developed by Ding and Spector [5]. GLS provides a method for us to evolve neural networks with LS. Given a population of neural

networks and some training data, GLS trains each network on a disjoint subset of the data using the SGD optimizer and performs standard LS on the population, using the instances of that data as the selection cases as demonstrated in Algorithm 2. The selected network is then duplicated to restore the size of the population and the process is repeated for some predefined number of generations.

Algorithm 2 Gradient Lexicase Selection

```
data ← The entire training dataset
candidates ← Population of p neural networks initialized with the same parameters
G ← Number of generations

for each generation in G do
    subsets ← p equal-size disjoint subsets of data obtained from random sampling without
              replacement

    Use SGD and Backpropagation to optimize each of the p candidates on each of the p
    subsets respectively

    parent ← LexicaseSelection(candidates, data)                    ▷ See Algorithm 1

    candidates ← Population of p neural networks initialized with the same parameters as
                 parent
end for

return parent
```

Due to computational constraints, we evolve a population of four models as do Ding and Spector. Furthermore, given that each Single-Task NCA is only trained on one task and each task only has about 3 input/output pairs describing the intended transformation, we opt to increase the number of selection cases by using the individual pixels of each example as the selection cases, as opposed to the full examples themselves. There are two ways in which we determine which NCA survives to the next round. The most straightforward of the two is to evaluate the correctness of each NCA's final prediction of the current pixel case. That is, after we have taken the $argmax$ of the final color probability distribution for that pixel, we directly compare that integer to the target integer

to determine if the target has been correctly predicted. Only those models which have correctly predicted the target color are selected to survive. We refer to this method as the Standard-LS approach. Another approach which may provide a finer level of discrimination between solutions is selecting over the post-$softmax$ probability distributions directly, allowing us to identify which NCA felt most confident that the target color was the right prediction to make. The motivation for such an approach is clear: If NCA A assigns a probability of 0.99 to the target color and NCA B only assigns it a 0.51 probability, both NCA, according to the Standard-LS approach, have correctly predicted the target. But clearly, NCA A was significantly more confident in its prediction and potentially exhibits more of the problem-solving behavior we are looking to develop. By evaluating our NCA in this way, we enforce a more granular selection criterion. This approach, however, encourages us to overhaul a small but consequential detail of the GLS algorithm. Just like our models, the models studied in Ding and Spector are multi-class classifiers. After each model is trained on its subset of the data, the selection phase of GLS selects only the models which make the correct classification on the given selection case. The issue with the confidence evaluation approach is that the predictions which the NCA make are continuous-valued, not discrete. Therefore, the NCAs are astronomically unlikely to share the same exact target color probability. This causes the algorithm to always choose the single NCA with the highest target color probability, eliminating competition within the population. This, however, is not a novel problem in the context of LS. La Cava, Spector, and Danai [11] devised a way to conduct LS in continuous-valued error spaces with Epsilon Lexicase Selection (ELS). ELS combats the improbability that two candidate solutions share the same score by defining an epsilon threshold from the best solution's score at every generation and allowing the solutions which score within that threshold to pass to the next round of selection. It is essentially a way of determining if a solution is "good enough" to survive. For example, if we choose an epsilon of 0.2 and assume that NCA A is the best performer, NCA B would not pass through to the next round as $0.51 \not> 0.99 - 0.2$. We explore three epsilon-determination methods. The standard approach, explored in [11], is choosing the median absolute deviation (MAD) of all the scores. We also explore choosing epsilon such that the best half of the population always gets through (referred to as the Best Half approach) as well as a fixed value for epsilon. For this fixed

value, we use 0.4 as it was shown by earlier work in this exact domain to work best compared to other values [2]. We alter another piece of GLS to ensure that each Single-Task NCA in the population has sufficient data to train on. Since we expect only about 3 examples per task, we cannot always allocate each model its own disjoint subset of the training data. To remedy this, we depart from the original implementation of GLS and allow each model to randomly sample, with replacement, twice the number of examples there are in the given task. So, at the cost of some population diversity, we ensure that each model receives substantial exposure to data. All configurations are evolved over 800 generations.

3.3 Multi-Task NCA

Training a single NCA on all of the ARC-AGI tasks as a solution to the benchmark poses a much more complex problem. This approach necessitates that the system, already constrained by its architectural design, further generalizes across *multiple* tasks. By expanding the target domain and maintaining a fixed architecture, we significantly amplify the difficulty of convergence to a successful solution. As such, we expect the performance of these models, which we refer to as Multi-Task NCA, to be very poor. Indeed, our baseline Multi-Task NCA solved only 1 out of the 172 tasks. We use the same approaches we did for evolving the Single-Task models as we do for these, plus an extra approach. On top of using individual pixels as selection cases with the various epsilon-determination methods, we also explore using the full input/output pairs themselves as the selection cases. Since these NCA are trained on all of the tasks, we are no longer restricted to only 3 input/output pairs, but instead have access to a total of 516 pairs across all tasks. For this approach, we continue to use ELS to determine which models survive each round, but instead of evaluating them by confidence, we look directly at the error between the current state of the given task and the target state. Additionally, given the surplus of training data available to these models, we can allocate disjoint subsets for each during the training phases of GLS without the data scarcity concern. That is, the GLS procedure is unchanged for experiments on Multi-Task NCA.

3.4 Fine-tuning with GLS

Approaching evolution for NCA from a different angle, we explore whether evolution is better fit as a model fine-tuning effort. By design, when a child is selected at a particular iteration of GLS, only its weights are copied over to its clones; no optimizer information is passed on to the next generation of NCA. When the training data is sufficiently large, this does not pose much of an issue as the optimizer is able to take many steps and significantly contribute towards learning every generation. However, the ARC-AGI tasks are designed with a small number of examples so as to require abstraction and generalization abilities from solutions. This means that when we train the Single-Task NCA, the optimizer walk is extremely short, hardly helping the models learn from the examples at all. This is only exacerbated by the tendency of our selection approaches to converge prematurely on a model. Our experiments show that the selection phases rarely get through more than 50 of the selection cases before choosing a winning model. When they do, it is because the selection technique has lost the granularity to filter out subpar models, failing to decrease the size of the population. The most extreme example of this is the Best Half epsilon-determination method. Since we evolve populations of 4 models, this approach selects a winner after only 2 cases. We posit that both Single and Multi-Task NCA may benefit from building foundational priors before being evolved with GLS. As a final study, we apply 200 generations of GLS to the trained baseline models, which have 800 epochs of uninterrupted optimization, as a fine-tuner. Table 3.1 provides a summary of all our experimental configurations.

Table 3.1: Summary of Experimental Configurations

Configuration Name	**Training/Evolution Approach**	**Selection Cases**	**Epsilon-Determination Methods**
Baseline Single-Task	SGD on a Single Task	—	—
Evolved Single-Task	GLS from scratch	Pixels	Standard-LS, $\epsilon = 0.4$, BH, MAD
Fine-tuned Single-Task	GLS from Baseline	Pixels	Standard-LS, $\epsilon = 0.4$, BH, MAD
Baseline Multi-Task	SGD on Every Task	—	—
Evolved Multi-Task	GLS from scratch	Pixels, Examples	Standard-LS (On Pixel Cases), $\epsilon = 0.4$, BH, MAD
Fine-tuned Multi-Task	GLS from Baseline	Pixels, Examples	Standard-LS (On Pixel Cases), $\epsilon = 0.4$, BH, MAD

Chapter 4

Results

4.1 Baseline NCA

We successfully replicated Xu and Miikkulainen's result of 13.37% (23/172) of tasks solved with the baseline Single-Task NCA. Alongside evaluating models by the proportion of the tasks which they solved, we also investigate per-task accuracy, calculated as the proportion of pixels in the final grid which were correctly predicted. The per-task accuracies of the Single-Task baseline models span a wide range but are consistently > 0.7, indicating strength in the NCA approach. Though the Multi-Task baseline model solved only one task, its accuracies also frequently fall above 0.7. As shown in Figure 4.1, the Single-Task baseline NCA completely fails on two tasks, while the Multi-Task baseline fails on three, sharing task 5582e5ca as a common failure. Interestingly, we found a considerable overlap in tasks which the Single-Task and Multi-Task baseline models performed well on and similarly for those which both models performed poorly on. Figure 4.1 shows that some of the tasks which saw performance from the Single-Task baseline NCA in the top 10% of accuracy were also in the top 10% for the Multi-Task NCA. Namely, they were tasks a699fb00, 32597951, dc433765, 5168d44c, b1948b0a, and 3618c87e. The shared tasks landing in the bottom 10% were tasks 68b16354, 363442ee, 6e01f1e3, 9565186b, 5582e5ca, 85c4e7cd, bd4472b8, 67a3c6ac, 05269061, and d511f180.

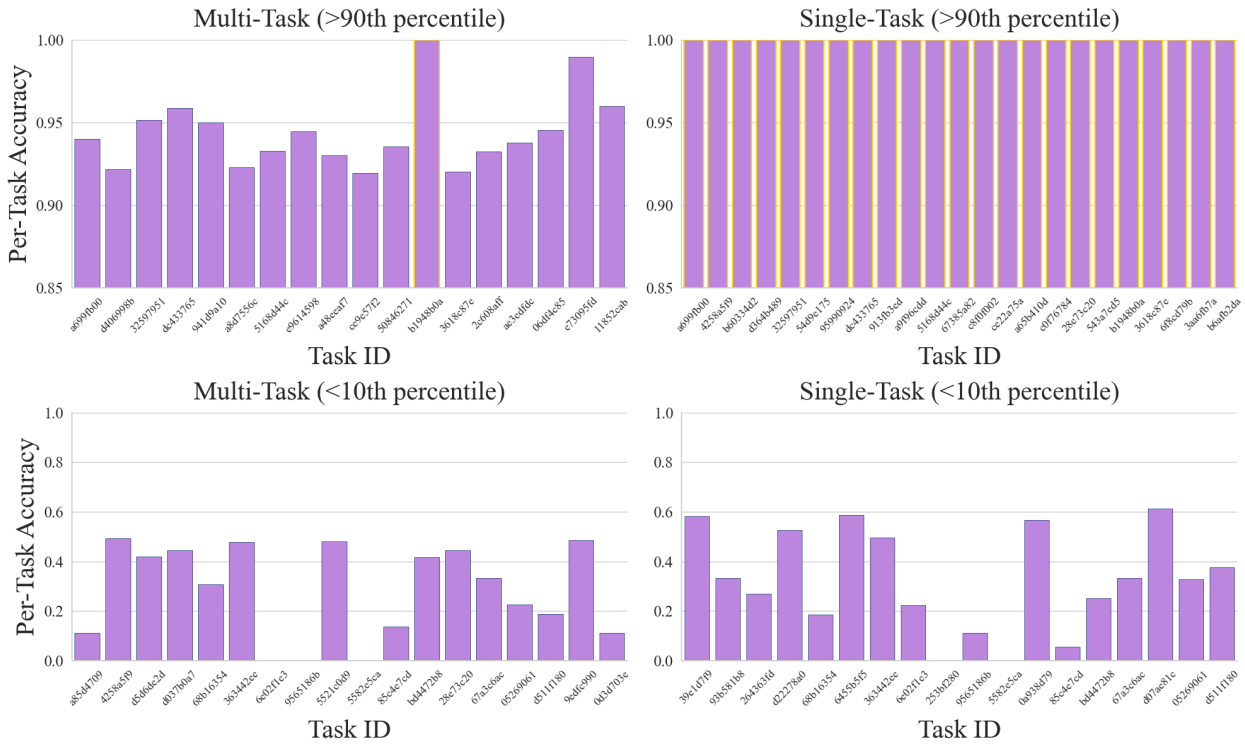

Figure 4.1: Per-Task Accuracy of Baseline Multi-Task and Single-Task NCA on Tasks Which Landed in the Top and Bottom Deciles of Performance with a Yellow Border Indicating a Solved Task

4.2 Evolved NCA

Evolving the Single-Task NCA with GLS degraded the performance over baseline across the board, though only by a small amount. Each of the three epsilon-determination methods, along with the Standard-LS approach, saw an average change in per-task accuracy of approximately -0.04. It is important to note that this average includes both tasks which were degraded and also those which were improved on, of which there were several for each configuration. Every configuration also solved at least one task which the baseline did not solve, yet none of them surpassed the 23/172 solved task rate of the baseline. The MAD epsilon-determination method achieved the least degradation in average per-task accuracy as shown in Table 4.1. Despite all four configurations degrading mean accuracy, the magnitude of degradation is modest, and the fact that every configuration solved at

least one task which the baseline did not suggests GLS is diversifying the solution space.

Table 4.1: Single-Task NCA: GLS Configurations vs. Baseline

Configuration	**Mean Per-Task Acc.**	**Mean Change in Acc.**	**# of Tasks Solved**	**# of Tasks Improved**	**# of Tasks Degraded**
Baseline	0.8299	—	23	—	—
$\epsilon = 0.4$	0.7865	−0.0433	16	40	87
Best Half	0.7835	−0.0463	12	33	93
MAD	0.7936	**−0.0363**	13	43	89
Standard-LS	0.7909	−0.0389	11	39	87

The result is mostly the same in the Multi-Task case with the exception of two configurations: Best Half using pixel selection cases and $\epsilon = 0.4$ using example selection cases. Each found a small improvement in average per-task accuracy of 0.0002 and 0.0085 respectively, as shown in Table 4.2. Though the $\epsilon = 0.4$ configuration performed better, the Best Half configuration was the only one to solve a task which the baseline model had not solved. However, it also failed to solve the task that the baseline originally solved, which the $\epsilon = 0.4$ configuration managed to preserve. The identical metrics shared across several Multi-Task configurations, namely $\epsilon = 0.4$ and MAD using pixel selection cases, Best Half and MAD using example selections, and Standard-LS, are not coincidental; these configurations converged on the same transformation strategies regardless of their selection criteria, collapsing what would otherwise be distinct evolutionary trajectories into a single behavioral outcome. The nature of these transformations is examined in Section 5.3.

4.3 Fine-tuned NCA

Fine-tuning the baseline Single-Task NCA yielded much more consistent improvement, though still small. Every configuration, across the Single and Multi-Task categories, achieved a positive change in the average per-task pixel accuracy over the baselines and degraded significantly fewer tasks than their counterparts. Two of the Single-Task models, namely the $\epsilon = 0.4$ and Standard-LS

Table 4.2: Multi-Task NCA: GLS Configurations vs. Baseline

Configuration	**Mean Per-Task Acc.**	**Mean Change in Acc.**	**# of Tasks Solved**	**# of Tasks Improved**	**# of Tasks Degraded**
Baseline	0.7487	—	1	—	—
$\epsilon = 0.4$ (Pixels)	0.5771	−0.1716	0	21	125
Best Half (Pixels)	0.7489	**+0.0002**	1	88	47
MAD (Pixels)	0.5771	−0.1716	0	21	125
$\epsilon = 0.4$ (Examples)	0.7572	**+0.0085**	0	90	44
Best Half (Examples)	0.5771	−0.1716	0	21	125
MAD (Examples)	0.5771	−0.1716	0	21	125
Standard-LS	0.5771	−0.1716	0	21	125

configurations, were able to solve a task which the baseline had not solved but which was already quite close to solving, resulting in a solve rate of 24/172 for these configurations. Across both categories, the Standard-LS approach dominated.

Table 4.3: Fine-tuned Single-Task NCA: GLS Configurations vs. Baseline

Configuration	**Mean Per-Task Acc.**	**Mean Change in Acc.**	**# of Tasks Solved**	**# of Tasks Improved**	**# of Tasks Degraded**
Baseline	0.8299	—	23	—	—
$\epsilon = 0.4$	0.8332	+0.0034	24	13	11
Best Half	0.8354	+0.0056	23	14	11
MAD	0.8362	+0.0063	23	15	9
Standard-LS	0.8377	**+0.0079**	24	19	10

Table 4.4: Fine-tuned Multi-Task NCA: GLS Configurations vs. Baseline

Configuration	**Mean Per-Task Acc.**	**Mean Change in Acc.**	**# of Tasks Solved**	**# of Tasks Improved**	**# of Tasks Degraded**
Baseline	0.7487	—	1	—	—
$\epsilon = 0.4$ (Pixels)	0.7569	+0.0082	1	94	41
Best Half (Pixels)	0.7556	+0.0069	1	85	40
MAD (Pixels)	0.7602	+0.0114	0	89	38
Standard-LS	0.7621	**+0.0134**	1	85	42
$\epsilon = 0.4$ (Examples)	0.7566	+0.0078	0	84	47
Best Half (Examples)	0.7554	+0.0066	0	84	43
MAD (Examples)	0.7546	+0.0058	1	84	40

Chapter 5

Discussion

5.1 GLS for Refinement

The consistently positive change in per-task accuracy from the fine-tuned models, among several other dominating metrics, supports the use of GLS as a refinement tool over a dedicated learning algorithm. Averaged across all the Single-Task configurations which were evolved from scratch, i.e., not fine-tuned, 74% of the ARC-AGI tasks saw a non-zero change in per-task accuracy from the models. Of these tasks, the models showed improvement on only 30% of them. The fine-tuned models, on the other hand, only affected the per-task accuracy of 15% of the tasks, but improved on roughly 60% of them. Importantly, none of the tasks which were degraded by the fine-tuned models were solved by the baseline models, resulting in the 24/172 solve rate. This trend persists for the Multi-Task models with the biggest difference being an increased proportion of tasks which saw a non-zero change in per-task accuracy from the fine-tuned models. The Multi-Task models evolved from scratch using pixel selection cases improved accuracy on 26% of the affected tasks while their fine-tuned counterparts improved 68% of them. Similarly, the evolved-from-scratch models using example selection cases found improvement on 31% of the affected tasks, while the fine-tuned variants improved on 65% of them. These proportions are visualized in Figure 5.1.

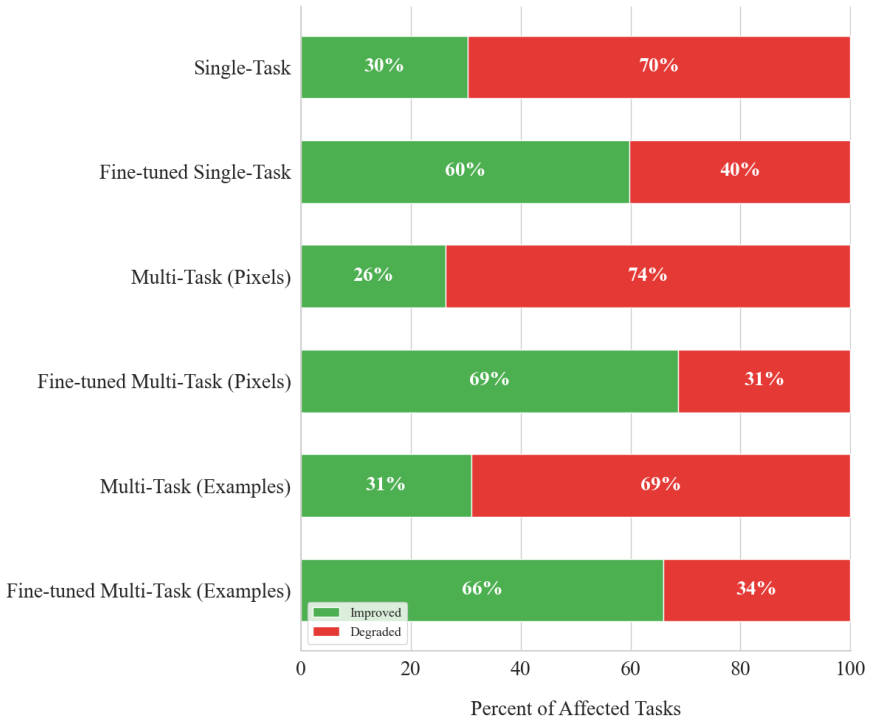

Figure 5.1: Proportion of Affected Tasks Improved and Degraded by Evolved and Fine-tuned Models, Averaged Across Epsilon-Determination Methods

5.2 Impact of Example Density on Performance

Though the evidence suggests crowning the fine-tuning approach as the winner of our experiments, we would be remiss not to acknowledge the large improvements the Single-Task models evolved from scratch made on the tasks which the baseline models did not fully solve. There are several such examples, but two tasks stand out as particularly impressive. The baseline model completely failed on task 253bf280, while each of the evolved models was able to achieve near-perfect performance on it. Less striking but still noteworthy is task 794b24be. The baseline model achieved a high but imperfect accuracy on this task while two of the evolved configurations found the correct solution. The other configurations found the same solution as the baseline. The common thread between these tasks, and other tasks on which the evolved models outperformed the baseline, is the number of training examples. As we have mentioned, ARC-AGI typically contains about 3 examples, but

task 253bf280 and task 794b24be contain 8 and 10 examples respectively. Our departure from disjoint sampling in the training phase of GLS for the Single-Task models has large implications for the amount of data these models have to train on. Recall that for each generation of our modified GLS algorithm, the Single-Task models sample, with replacement, twice the number of examples there are in the given task. This means that instead of an average of 6 examples to train on in any given generation, the models trained on these example-dense tasks have anywhere from 10 to 20 examples available to them. Across all epsilon-determination methods, we found a statistically significant correlation (p-value < 0.02) between the number of examples given in a task and a Single-Task model's average accuracy improvement over baseline, demonstrated in Figure 5.2.

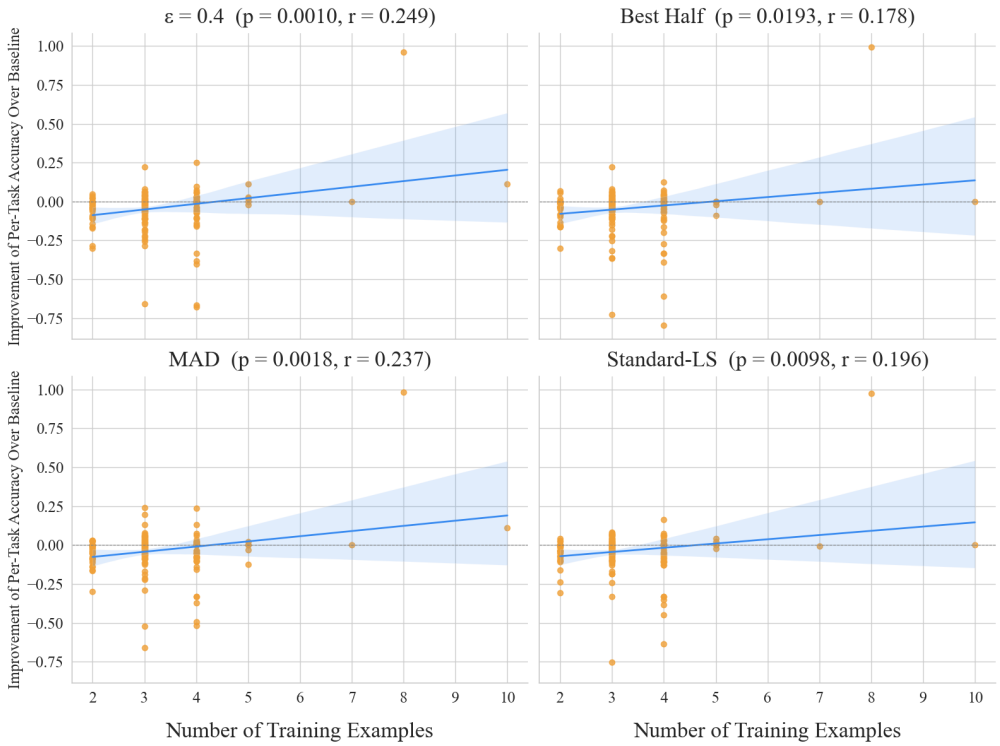

Figure 5.2: Correlation Between Example Density and Improvement over Baseline of Single-Task Configurations, p Denoting the p-value and r Denoting the Pearson Correlation Coefficient

This result is consistent with the original design of GLS. GLS is a two-part system: The effectiveness of its evolutionary component is strictly dependent on the quality of the signal produced

during its gradient-based training phase. When training examples are scarce, as is the case in many of the ARC-AGI tasks, the gradient updates calculated in each generation are likely noisy and underdeveloped. This leads to a selection process which primarily filters on stochasticity rather than meaningful divergence. Conversely, as the number of examples per task increases, each generation's update becomes more representative of the underlying task logic. This result also offers a theoretical explanation for why fine-tuning with GLS outperformed evolution from scratch. The trained baseline models already possess robust representational priors. Therefore, even a small training signal per generation is adequate to produce selectable differences between solutions. In contrast, the models evolved from scratch require the training phase to simultaneously build foundational representations and produce diversified variation. This dual burden appears unsustainable when task examples are limited. Practically, these results suggest that the effectiveness of GLS is contingent on sample density. The larger implication is that GLS is most effective in data-rich domains or, alternatively, that it encourages a departure from disjoint sampling to ensure that its training phase remains informative enough to support the subsequent evolutionary selection.

5.3 Behavior

Our experiments evolving NCA from scratch with GLS resulted in many instances of improvement over baseline performance, and though this is promising, we want to ensure that these improvements are the result of non-trivial, task-relevant behavior. Indeed, there are a few tasks on which the evolved Single-Task models showcase a tendency towards prioritizing cheap heuristics. In these cases, the most prevalent strategies were either to replicate the input grid exactly or to output a fully black grid. Many of the ARC-AGI tasks use the color black as a background to highlight the intended transformation, and oftentimes, the pixels which are actually relevant to the transformation make up a small proportion of the total pixels. It is reasonable, then, that a model might overfit to the high frequency of black pixels in the tasks and converge on the safe but suboptimal behavior of turning every pixel black. For the most part, however, the Single-Task models did not fall for such cheap heuristics. The baseline Single-Task NCA only exhibited this kind of

behavior on two tasks, and none of the evolved variants were able to deter it from that direction. The Multi-Task models were dramatically more susceptible to getting stuck in this local optimum. Similar to the Single-Task NCA, the Multi-Task baseline model almost never converged on the aforementioned strategies. However, the evolved variants overwhelmingly favored them. In fact, the evolved Multi-Task NCA rarely predicted a grid that was not a result of one of these strategies, highlighting the strength of this bias. A visualization of the Multi-Task configurations converging on these strategies is provided in Figure 5.3. The fine-tuned Single-Task models mostly inherited the behavior of their parent baseline models and often did not veer from the baselines' predictions. When they did predict a grid different from the baseline's, their behavior demonstrated a shift toward more complex, task-relevant transformations, potentially suggesting an escape from the local optima trap via fine-tuning. The fine-tuned Multi-Task models also showed a smaller tendency towards adopting the cheap strategies, primarily favoring small alterations to the baseline predictions.

It is not particularly surprising that our NCA learned these behaviors. As we touched on, the prevalence of black in the ARC-AGI tasks is an overwhelming statistical bias which the models are prone to exploit. Furthermore, the architecture of our NCA makes the identity transformation especially cheap to learn given that the pre-training state of any given task is already the product of said transformation. The asynchronicity of the NCA further protects their adherence to the initial state. By design of the asynchronous update mask, a substantial fraction of cells may not update at all on any given step. This means that even if the learned update rule begins to diverge away from the identity transformation, the stochastic updating dampens the effect and keeps the output close to the input. This may suggest that selection granularity alone cannot solve generalization. When such cheap strategies are competitive case by case, as with input-replication with high input/output overlap or the all-black transformation with black-dominated outputs, the per-case filtering of LS cannot discriminate them from task-relevant solutions.

A large portion of all the configurations of NCA we explored in our experiments resisted the cheap strategies on the tasks on which, reported in Section 4.1, both the Single-Task and Multi-Task baseline NCA found shared success. For the Single-Task models, an impressive 0% of all

configurations resorted to the all-black or identity transformations on these tasks. Though the same cannot be said of the Multi-Task models, the configurations for which we reported a positive change in mean per-task accuracy (Best Half using pixel selection cases and $\epsilon = 0.4$ using example selection cases, as seen in Table 4.2) managed to evade convergence on these strategies on all tasks except task b1948b0a. Here, both configurations were the only ones to opt for the identity transformation while the others took the all-black approach. This behavior, coupled with our finding that NCA perform well on the same ARC-AGI tasks invariant of how we train them, appears to suggest some notion of objectively easy and hard tasks for NCA. A qualitative analysis of the design of the tasks on which both variants of baseline models found shared success led us to define categories by which we can group and thus characterize them. We were able to similarly define categories for the tasks on which the baseline models found shared failure.

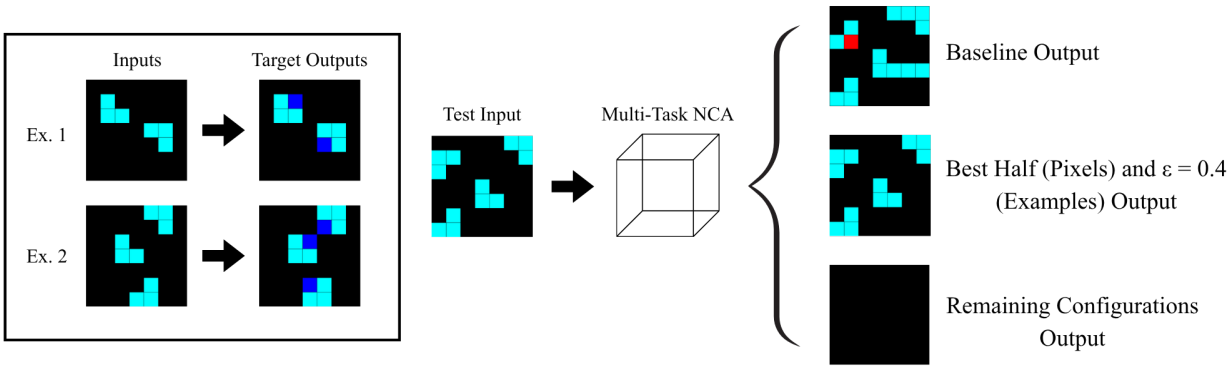

Figure 5.3: Outputs of all Multi-Task Configurations on Task 3aa6fb7a (Examples Shown on the Left)

We identified two categories for the shared-success tasks. Tasks a699fb00, 32597951, and b1948b0a are simple pixel mapping tasks. These tasks require the solver to locate pixels of a common attribute and subsequently change them all to the same color. Tasks dc433765, 5168d44c, and 3618c87e require select pixels to be moved short distances, leaving the rest of the grid unchanged. For the shared-failure tasks, we identified two more categories: These were tasks requiring solutions

to count particular features of tasks, e.g., colors or shapes, and tasks where the transformation of particular pixels is determined by far-away pixels. All categories are demonstrated by Figure 5.4.

It appears, then, that the category in which each task belongs has a direct impact on the ability of our NCA architecture to solve them. Beyond simple performance metrics, these categories reveal an intrinsic relationship between the local dynamics of NCA and their global reasoning capabilities. Tasks where global transformations can be decomposed into uniform, local symmetries like color-mapping or short-distance movement align with how CA update rules are defined. In contrast, tasks which require a departure from local dependencies seem to be unnatural to the architecture of CA. This supports the hypothesis that systems defined by local dynamics face a structural hurdle when tasked with making abstract, non-local generalizations. The architectural basis for the distinction between easy and hard tasks becomes clear when we revisit the mechanics of the convolutional structure of NCA. Recall that the 3×3 convolutional filters of our NCA restrict each cell's perception to its immediate neighborhood at every time-step. For tasks in the pixel mapping and short-distance movement categories, this locality is not a limitation; the transformation of any given pixel can be determined entirely by inspecting its nearby neighbors, which is precisely what the 3×3 filters are designed to do. Long-distance dependence tasks, however, require a cell to effectively "know" about the state of pixels that may be many cells away. In principle, an NCA could propagate information across the grid over many time-steps, with each step relaying information one neighborhood further. In practice, however, this kind of long-range signaling is difficult to learn reliably, particularly under the asynchronous updating regime we employ. Since cells update stochastically, the propagation of distant signals is noisy and inconsistent, making it unlikely that a stable long-range communication channel ever develops. Counting tasks present a distinct but related challenge. To count particular features of a task, a system must accumulate a global state that reflects the entire grid, not just local neighborhoods. This requires something analogous to memory that persists and updates coherently across the full spatial extent of the grid. The hidden channels of the NCA offer a potential substrate for this, but learning to use them as a reliable global accumulator through local interactions alone is a fundamentally difficult optimization problem, particularly when training examples are scarce. Together, these two failure

categories not only reveal empirical limitations of our models, but underline the structural tension at the heart of applying locally-defined systems to globally-reasoning tasks.

We did not find that any of our evolutionary methods improved performance when considering tasks of any particular category. This may suggest that, in order to optimize the efficacy of evolutionary approaches on NCA applied to ARC-AGI, selecting for task-specific behavior may yield greater performance. In the context of GLS, this most naturally alludes to re-evaluating how the selection cases are defined. The specialized re-definition of selection cases, however, introduces a significant degree of human intervention. This risks shifting the approach of NCA for ARC-AGI from emergent learning toward program synthesis; in doing so, we may inadvertently hand-hold the NCA toward solutions rather than allowing the evolutionary process to discover them autonomously, which is the principal motivation of our work.

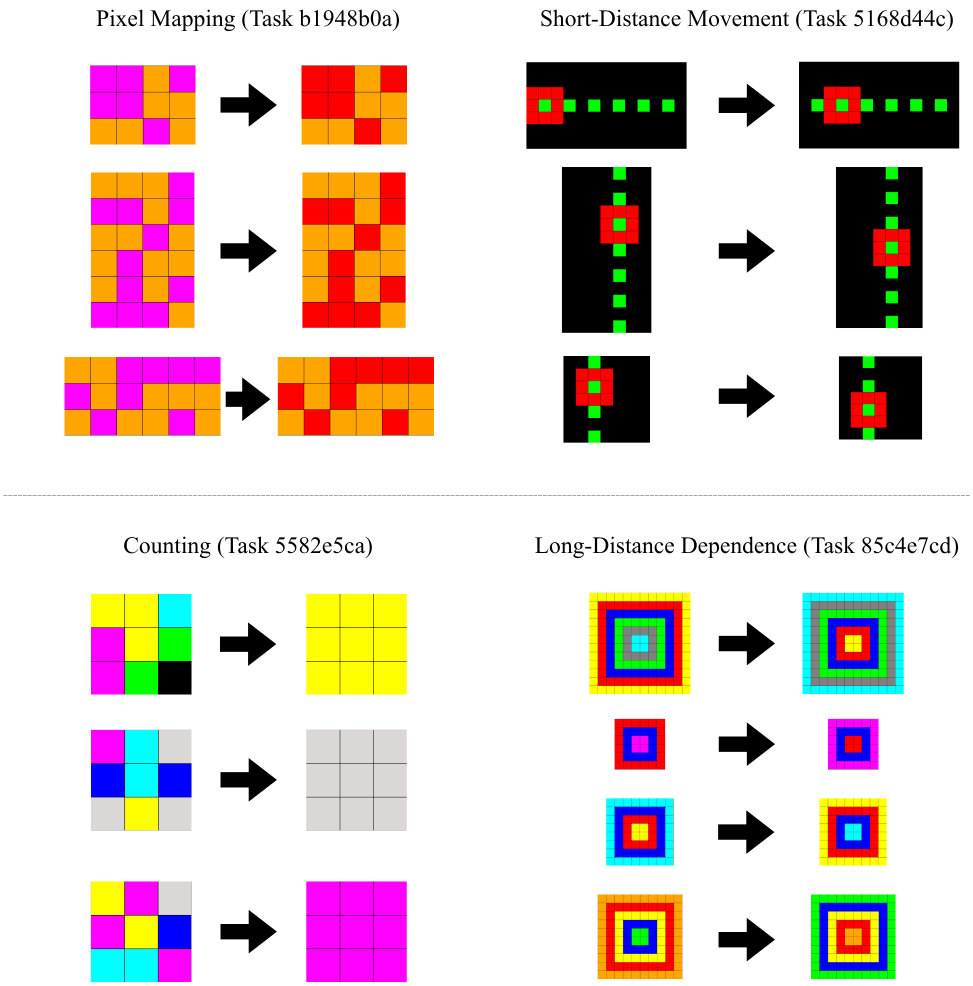

Figure 5.4: Training Examples of Tasks in Each Category

Chapter 6

Conclusion and Future Work

Nature proves that systems defined by local dynamics are capable of complex and intelligent behavior. Our application of NCA to ARC-AGI serves as a method for studying how similar systems can be designed and improved to achieve these behaviors in a computational setting. In particular, our approach investigated whether the evolutionary processes which have developed these natural systems play a significant role in the emergence of their intelligence. We centered our evolutionary approaches around Lexicase Selection, which is known to excel in multi-objective domains such as ARC-AGI. Towards the promotion of population diversity, we explored several approaches for determining which solutions are most fit via our epsilon-determination methods. Though we cannot say that our application of evolution to NCA definitively encouraged more intelligent behavior, our experiments resulted in a number of suggestive findings.

Our use of GLS as a refinement tool highlighted the critical role of pre-existing representational priors in NCA. These experiments demonstrated that the evolutionary process was effectively able to polish task-specific logic and positioned GLS as a stronger fine-tuner than a from-scratch optimizer in the ARC-AGI domain. This result is deeply linked to our findings regarding example density. We observed a statistically significant correlation between the number of training examples provided in a task and the ability of GLS to improve performance over NCA trained with conventional, non-evolutionary methods. GLS assumes the two-part responsibility of developing foundational features and diversifying logic. We found that the scarcity of data per task in ARC-AGI inherently

clashed with the design of the training portion of GLS, suggesting that this dual responsibility is not sustainable for data-sparse domains. Furthermore, modifications to GLS that increase the amount of data which the NCA are trained on in each generation may provide a path towards evolving better-performing models.

Our primary investigation of GLS as a from-scratch optimizer for NCA led us to discover a bias towards trivial behavior across many configurations. We found that in order for evolutionary approaches to be effective in the ARC-AGI tasks, they must be robust to overfitting on deceptive signals such as color frequency and input/output similarity. Tasks in which several configurations of the evolved NCA found shared success saw a significantly lower frequency of convergence on such behavior. This finding led us to formalize a notion of easy and hard tasks for NCA. We identified qualitative features of tasks which were consistently associated with particularly strong and poor behavior, underlining the inherent challenge of using systems defined by local interactions for tasks requiring global coordination, as the hard tasks do.

Our work thus puts forward two major directions from which we can approach the further development of NCA as an ARC-AGI solver in future work. The most apparent and actionable of these approaches is investigating alternative NCA architectures and evolutionary algorithms. The NCA architecture we studied in this thesis is very niche in the sense that, considering its thousands of hyperparameters, it represents only a single point in a massive high-dimensional parameter space. Our work did not explore, for example, the relationship between the number of hidden channels and accuracy. Experiments exploring many such relationships could be hugely informative to our results. However, as is well-known in the field of machine learning, converging on a set of hyperparameters for neural network architectures is difficult. Neural networks are defined by various features, each of which can have a potentially infinite number of parameters. This combinatorial explosion is apparent even in the small models we studied in this work. Future work might explore Evolutionary Neural Architecture Search towards building a stronger NCA foundation for ARC-AGI. In the same vein, exploring new modifications to GLS as well as alternate evolutionary algorithms altogether which place a stronger emphasis on training could be fruitful, as evidenced by the correlation we found.

The second approach to improving NCA for ARC-AGI involves the re-evaluation of how we define the selection cases in the LS procedure. In place of evaluating NCA by their error on individual pixels or full examples, rewarding abstract, higher-level behaviors such as symmetry preservation and object permanence may guide models towards task-relevant logic and away from the cheap heuristics which pervade the ARC-AGI solution search space. Curriculum Learning, a training strategy which incrementally scales the difficulty of data on which neural networks are trained, presents a compelling way forward in this context. By first anchoring the population on the tasks which we were able to categorize as "easy" and gradually introducing the harder tasks, we can steer the evolutionary trajectory toward genuine abstract reasoning. Furthermore, incorporating Novelty Search or behavioral diversity metrics into the selection process could prevent the population from stagnating in trivial local optima. In general, rewarding the discovery of unique and task-relevant transformation rules may provide the necessary pressure for NCA to transcend their local biases and achieve the global-level intelligence required by the ARC-AGI benchmark.

Bibliography

[1] Cesar Ascencio-Piña et al. "Image segmentation with Cellular Automata". In: *Heliyon* 10.10 (May 2024), e31152. ISSN: 2405-8440. DOI: 10.1016/j.heliyon.2024.e31152. URL: http://dx.doi.org/10.1016/j.heliyon.2024.e31152.

[2] Ivan Betancourt and Lee Spector. *Evolving Neural Cellular Automata with Gradient Lexicase Selection*. Poster at The Genetic and Evolutionary Computation Conference (GECCO). To appear. 2026.

[3] Tobias Blickle and Lothar Thiele. *A Comparison of Selection Schemes used in Genetic Algorithms*. Tech. rep. 11. Version 2 (2nd Edition). TIK, 1995.

[4] François Chollet. *On the Measure of Intelligence*. 2019. arXiv: 1911.01547 [cs.AI]. URL: https://arxiv.org/abs/1911.01547.

[5] Li Ding and Lee Spector. *Optimizing Neural Networks with Gradient Lexicase Selection*. 2023. arXiv: 2312.12606 [cs.LG]. URL: https://arxiv.org/abs/2312.12606.

[6] Martin Gardner. "MATHEMATICAL GAMES". In: *Scientific American* 223.4 (1970), pp. 120–123. ISSN: 00368733, 19467087. URL: http://www.jstor.org/stable/24927642 (visited on 03/31/2026).

[7] William Gilpin. "Cellular automata as convolutional neural networks". In: *Phys. Rev. E* 100 (3 Sept. 2019), p. 032402. DOI: 10.1103/PhysRevE.100.032402. URL: https://link.aps.org/doi/10.1103/PhysRevE.100.032402.

[8] Etienne Guichard et al. *ARC-NCA: Towards Developmental Solutions to the Abstraction and Reasoning Corpus*. 2025. arXiv: 2505.08778 [cs.AI]. URL: https://arxiv.org/abs/2505.08778.

[9] Etienne Guichard et al. *EngramNCA: a Neural Cellular Automaton Model of Memory Transfer*. 2025. arXiv: 2504.11855 [cs.NE]. URL: https://arxiv.org/abs/2504.11855.

[10] Thomas Helmuth, Lee Spector, and James Matheson. "Solving Uncompromising Problems With Lexicase Selection". In: *IEEE Transactions on Evolutionary Computation* 19.5 (2015), pp. 630–643. DOI: 10.1109/TEVC.2014.2362729.

[11] William La Cava, Lee Spector, and Kourosh Danai. "Epsilon-Lexicase Selection for Regression". In: *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. GECCO '16. ACM, July 2016, pp. 741–748. DOI: 10.1145/2908812.2908898. URL: http://dx.doi.org/10.1145/2908812.2908898.

[12] Genaro J. Martinez, Harold McIntosh, and Juan Seck Tuoh Mora. "Reproducing the Cyclic Tag System Developed by Matthew Cook with Rule 110 Using the Phases f(i-)1". In: *J. Cellular Automata* 6 (Jan. 2011), pp. 121–161.

[13] Giorgia Migliaccio et al. "Exploring Cell Migration Mechanisms in Cancer: From Wound Healing Assays to Cellular Automata Models". In: *Cancers* 15 (2023). URL: https://api.semanticscholar.org/CorpusID:265024385.

[14] Brad L. Miller and David E. Goldberg. "Genetic Algorithms, Tournament Selection, and the Effects of Noise". In: *Complex Syst.* 9 (1995). URL: https://api.semanticscholar.org/CorpusID:6491320.

[15] A. Mordvintsev et al. "Growing Neural Cellular Automata". In: *Distill* (2020). URL: https://api.semanticscholar.org/CorpusID:213719058.

[16] Paul Rendell. *Turing Machine Universality of the Game of Life*. Springer International Publishing, 2016. ISBN: 9783319198422. DOI: 10.1007/978-3-319-19842-2. URL: http://dx.doi.org/10.1007/978-3-319-19842-2.

[17] Sasmitha. *Efficiently Solving ARC-AGI problems with Evolved Neural Cellular Automata (ENCA)*. https://enca-arc.mmzdev.com/. Accessed: 2026-04-06. 2025. URL: https://enca-arc.mmzdev.com/.

[18] Lee Spector. "Assessment of problem modality by differential performance of lexicase selection in genetic programming: a preliminary report". In: *Proceedings of the 14th Annual Conference Companion on Genetic and Evolutionary Computation*. GECCO '12. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 2012, pp. 401–408. ISBN: 9781450311786. DOI: 10.1145/2330784.2330846. URL: https://doi.org/10.1145/2330784.2330846.

[19] Lee Spector, Li Ding, and Ryan Boldi. *Particularity*. 2023. arXiv: 2306.06812 [cs.NE]. URL: https://arxiv.org/abs/2306.06812.

[20] Christian Szegedy et al. *Going Deeper with Convolutions*. 2014. arXiv: 1409.4842 [cs.CV]. URL: https://arxiv.org/abs/1409.4842.

[21] Darrell Whitley. "A genetic algorithm tutorial". In: *Statistics and Computing* 4.2 (June 1994). ISSN: 1573-1375. DOI: 10.1007/bf00175354. URL: http://dx.doi.org/10.1007/BF00175354.

[22] Stephen Wolfram. "Tables of Cellular Automaton Properties". In: *Cellular Automata and Complexity*. Originally published 1986. CRC Press, 2018, pp. 513–584.

[23] Kevin Xu and Risto Miikkulainen. *Neural Cellular Automata for ARC-AGI*. 2025. arXiv: 2506.15746 [cs.NE]. URL: https://arxiv.org/abs/2506.15746.

[24] Biswarup Yogi, Ajoy Kumar Khan, and Satyabrata Roy. "Cellular automata based key distribution for lightweight hybrid image encryption with elliptic curve cryptography". In: *Scientific Reports* 15.1 (Oct. 2025). ISSN: 2045-2322. DOI: 10.1038/s41598-025-17536-7. URL: http://dx.doi.org/10.1038/s41598-025-17536-7.